

A Complete Rule Library

The proposed framework employs a predefined rule library for task-level settlement. The LLM does not generate new reward functions; instead, it selects from a fixed set of admissible settlement rules according to the observed task-completion context.

Table 1 summarizes all rules currently implemented in the framework. For brevity, abbreviated rule identifiers are used in the following tables. The corresponding implementation names are available in the released source code.

Table 1: Complete settlement rule library.

Rule ID	Primary Signal	Purpose
NONE	None	Disable task-level settlement.
CWP	Overlap + latency	Penalize congestion and delayed completion.
COP	Overlap	Penalize redundant participation.
WOP	Latency	Penalize prolonged task execution.
CWQ	Overlap + latency	Amplify severe contention.
CLP	Overlap + latency	Bound extreme negative settlement.
ARB	Latency + activity	Reward recovery of neglected tasks.
SAP	Progress stage	Adjust settlement according to episode progress.
PSP	Peak overlap	Penalize short-term crowding spikes.
ER	Activity efficiency	Encourage efficient task engagement.
FFB	Latency	Reward rapid task completion.
LCB	Episode progress	Encourage completion of remaining tasks.
TAR [†]	Global average	Uniformly distribute settlement among agents.

Abbreviations: CWP = crowd_wait_penalty_v1; COP = crowd_only_penalty_v1; WOP = wait_only_penalty_v1; CWQ = crowd_wait_quadratic_v1; CLP = capped_linear_v1; ARB = abandon_rescue_bonus_v1; SAP = stage_adaptive_v1; PSP = peak_sensitive_v1; ER = efficiency_ratio_v1; FFB = fast_finish_bonus_v1; LCB = late_stage_completion_bonus_v1; TAR = team_avg_v1.

[†] Used only as an experimental baseline and not as a standard routing target.

[†] Used only as an experimental baseline and not as a standard routing target.

A.1 Context Variables

For a completed task, the settlement engine constructs a task context containing the following statistics:

- overlap_{avg} : average number of concurrent participants during task execution.
- overlap_{peak} : maximum number of concurrent participants observed during execution.

- latency: number of environment steps elapsed between task claiming and task completion.
- active_steps: number of steps during which at least one agent actively pursued the task.
- remaining_ratio: fraction of unfinished tasks at the completion event.

A.2 Representative Rule Definitions

The crowd-wait penalty is the default settlement rule:

$$f_{\text{crowd-wait}} = -\alpha (\text{overlap}_{\text{avg}} + \beta \text{overlap}_{\text{peak}}) - \lambda \text{latency}. \quad (1)$$

The capped-linear rule constrains excessive penalties:

$$f_{\text{capped}} = \max(f_{\text{crowd-wait}}, -C). \quad (2)$$

The fast-completion rule rewards rapid task completion:

$$f_{\text{fast}} = f_{\text{crowd-wait}} + \frac{\eta}{1 + \text{latency}}. \quad (3)$$

All remaining rules are implemented as deterministic functions of the same context variables and are available in the released source code.

B Deterministic Routing and LLM-Assisted Selection

B.1 Routing Pipeline

The settlement engine follows a hierarchical routing procedure. When a task completion event occurs, the environment first evaluates a set of deterministic routing conditions. Clearly identifiable cases are resolved directly without invoking the LLM. Only ambiguous completion contexts are forwarded to the LLM selector.

The routing process consists of three stages:

1. Deterministic routing.
2. Candidate rule selection.
3. Constrained LLM-assisted rule selection.

The selected rule is subsequently evaluated by the settlement engine to produce the task-level settlement signal.

B.2 Deterministic Routing Rules

Several high-confidence execution patterns are handled directly through deterministic routing.

Table 2: Representative deterministic routing conditions.

Condition	Selected Rule	Purpose
$\text{overlap}_{peak} \geq 3$ or $\text{overlap}_{avg} \geq 1.2$	<code>crowd_only_penalty_v1</code>	Severe contention
$\text{latency} \geq 300$ and $\text{overlap}_{avg} \leq 0.3$	<code>wait_only_penalty_v1</code>	Long delay
$\text{remaining_ratio} \leq 0.2$	<code>late_stage_completion_bonus_v1</code>	Late-stage completion

B.3 LLM Routing Constraints

The LLM is not permitted to generate settlement functions or reward values.

Instead, the LLM receives:

- task-completion context;
- auxiliary execution statistics;
- a predefined admissible rule set.

The output of the LLM is restricted to selecting exactly one rule from the admissible candidate set.

Consequently, all settlement functions remain fully deterministic and are defined independently of the LLM.

B.4 Prompt Structure

The prompt consists of four components:

1. System instruction.
2. Task-completion context.
3. Candidate rule descriptions.
4. Output-format specification.

The output format is restricted to a structured JSON object containing the selected rule identifier and confidence estimate.

A simplified example is shown below:

Select exactly one rule from the admissible candidate set.

Return only a valid JSON object.

Candidate rules:

- `crowd_only_penalty_v1`
- `wait_only_penalty_v1`
- `fast_finish_bonus_v1`

B.5 Computational Safeguards

To reduce computational overhead and improve routing stability, the implementation incorporates several safeguards:

- Deterministic routing before LLM invocation.
- Episode-level LLM budget constraints.
- Minimum interval between consecutive LLM calls.
- Response caching.
- Sticky-mode reuse when repeated contexts occur.
- Fallback to deterministic rules when LLM routing is unavailable.

B.6 Runtime Configuration

Table 3: LLM routing configuration.

Parameter	Value
Model	Qwen3.6-Plus
Temperature	0.0
Maximum output tokens	180
Timeout	6 s
Cache TTL	5 s
Minimum call interval	0.5 s
LLM-enabled training phase	First 30% episodes

C Environment and Training Configuration

C.1 Multi-Agent Task Allocation Environment

Experiments are conducted in a custom multi-agent task allocation (MATA) environment implemented using the OpenAI Gym interface.

The environment contains a set of mobile agents and a set of spatially distributed tasks. Each agent repeatedly selects a task from its action space and navigates toward the corresponding target location.

A task is considered completed when an agent reaches the task location. Task completion triggers the event-driven settlement mechanism described in Section 4.

Multiple agents may simultaneously pursue the same task. Such behavior creates execution contexts involving contention, waiting, and redundant participation, which are explicitly recorded by the settlement engine.

C.2 Observation and Action Space

Each observation contains the spatial state of all agents and tasks.

Agent nodes encode:

- position coordinates;
- occupancy status;
- assigned task information.

Task nodes encode:

- position coordinates;
- completion status;
- contention-related statistics.

The action space consists of task-selection decisions. An additional idle action is provided when no suitable task is selected.

For an environment containing N_t tasks, the action-space size is

$$|\mathcal{A}| = N_t + 1.$$

C.3 Environment Configuration

Table 4: Environment configuration.

Parameter	Value
Map size	50×50
Number of agents	5 / 10 / 20
Number of tasks	20 / 40 / 60
Action space	$N_t + 1$
Episode length	300
Task placement	Uniform random
Agent placement	Uniform random
Task completion	Reach target location

C.4 Environment Reward Structure

The environment reward is independent of the settlement mechanism.

Each agent receives:

- task-completion reward;
- movement penalty;
- contention penalty;
- idle penalty;
- progress reward.

The task-level settlement signal is computed separately and injected only when a task completion event occurs.

C.5 Settlement Configuration

Table 5: Settlement configuration.

Parameter	Value
Overlap coefficient (α)	0.2
Latency coefficient (λ)	0.001
Peak-overlap weight (β)	0.5

C.6 Training Hyperparameters

Table 6: Training hyperparameters of the evaluated MARL backbones.

Parameter	MASAC	MAPPO
Learning rate	5×10^{-7}	1.5×10^{-6}
Discount factor γ	0.95	0.95
Batch size	512	–
Replay buffer size	10^6	–
Entropy coefficient	Automatic tuning	0.01
Hidden dimension	256	64
PPO clip ratio	–	0.2
Update epochs	–	10
Training episodes	2000	2000
Random seeds	8	10

D Statistical Analysis and Additional Results

D.1 Evaluation Protocol

All experiments are conducted using multiple random seeds.

For each training run, the final return is computed as the average environment utility over the last 200 episodes of training.

The reported mean and standard deviation are calculated across independent random seeds.

MASAC-based experiments are evaluated using eight random seeds, whereas MAPPO-based experiments are evaluated using ten random seeds.

D.2 Statistical Significance Test

Performance differences are evaluated using a two-sided Welch’s t -test.

Given two independent samples

$$X = \{x_1, \dots, x_n\}$$

and

$$Y = \{y_1, \dots, y_m\},$$

the test statistic is

$$t = \frac{\bar{x} - \bar{y}}{\sqrt{\frac{s_x^2}{n} + \frac{s_y^2}{m}}},$$

where $\bar{x}$ and $\bar{y}$ denote sample means and s_x^2 and s_y^2 denote sample variances.

Welch’s approximation is used for the effective degrees of freedom. All reported p -values are two-sided.

D.3 MASAC-Based Results

Table 7: Statistical comparison of MASAC-based variants.

Comparison	Mean Difference	p -value	Significant
Ours vs MASAC	58.62	< 0.001	Yes
Ours vs w/o LLM	26.10	0.0024	Yes
Ours vs Team Reward	70.51	< 0.001	Yes

D.4 MAPPO-Based Results

Table 8: Statistical comparison of MAPPO-based variants.

Comparison	Mean Difference	p -value	Significant
Ours + MAPPO vs MAPPO	40.98	< 0.001	Yes

D.5 Seed-Level Final Returns

Table 9 reports the final-return values used for the MASAC-based statistical analysis.

Table 9: MASAC-based seed-level final returns.

Method	S1	S2	S3	S4	S5	S6	S7	S8
Ours	354.22	356.94	344.58	374.95	350.54	365.65	375.60	378.58
w/o LLM	316.73	350.88	326.42	354.34	339.46	315.39	345.80	343.23
Team Reward	278.84	284.31	288.48	287.34	298.98	321.49	287.70	289.85
MASAC	273.39	299.06	276.59	349.21	301.94	300.75	314.98	316.14

Table 10: MAPPO-based seed-level final returns.

Method	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10
MAPPO	312.97	358.74	307.93	334.30	351.92	320.77	284.09	319.02	331.88	308.21
Ours+MAPPO	331.18	356.69	376.35	362.56	373.61	345.09	380.22	375.11	389.30	349.48

D.6 Additional Notes

All statistical analyses are performed using the final-return values reported in Tables 9 and 10.

No post-hoc filtering or seed selection is applied. All completed runs are included in the reported statistics.